

OFFICIAL ARCHITECTURE & ENGINEERING DOCUMENTATION

Smart School Management System

Complete System Architecture, Flowcharts, Database Entity-Relationship Specification & Module Index

PROJECT VERSION

v1.5 (Production Ready)

RELEASE DATE

September 29, 2026

TECHNOLOGY STACK

Vanilla JS SPA + CodeIgniter 3.x + MySQL 8.x

MODULES & TABLES

45 Modules / 56 Relational Tables

DEVELOPED BY

Infosof Technologies

TARGET DEPLOYMENT

cPanel Shared Hosting / Linux VPS

Smart School Management System

Comprehensive Project Report

Infosof Technologies | Academic Year 2026–27

[!IMPORTANT] **Project:** Smart School Management System

Version: 1.5 (September 2026)

Stack: HTML/CSS/Vanilla JS (SPA) + CodeIgniter 3.x (REST API) + MySQL 8.x

Developer: Infosof Technologies

Institution: Smart School International, Kolkata

Table of Contents

1. [Executive Summary](#)
 2. [Technology Stack](#)
 3. [System Architecture](#)
 4. [Application Working Flow](#)
 5. [Module Flowcharts](#)
 6. [Database ER Diagram](#)
 7. [Role-Based Access Control \(RBAC\)](#)
 8. [API Architecture](#)
 9. [Module Catalogue \(45 Modules\)](#)
 10. [Key Design Decisions](#)
 11. [Deployment Architecture](#)
-

1. Executive Summary

Smart School is a **full-stack, web-based school management system** that digitises and centralises all administrative, academic, financial, and operational workflows of a school. It supports **six user roles** (Super Admin, Admin, Teacher, Accountant, Parent, Student), provides a **REST JSON API backend** powered by CodeIgniter 3, and a **SPA (Single Page Application) frontend** written in plain HTML/CSS/JavaScript.

Key Highlights

METRIC	VALUE
Total Modules	45
Database Tables	56
User Roles	6 (Super Admin, Admin, Teacher, Accountant, Librarian, Receptionist, Parent, Student)
API Endpoints	45+
Payment Gateway	Razorpay (Online + Offline)
Attendance Modes	Manual + QR Code Scan
Deployment Target	cPanel Shared Hosting / VPS

2. Technology Stack

TECHNOLOGY STACK	
Layer	Technology
Frontend UI	HTML5, Vanilla CSS3, Vanilla JavaScript (ES2020)
SPA Router	Hash-based routing (#/path) in app.js
State Store	window.SS_STORE (localStorage reactive engine)
RBAC	window.canManage() – token-based role check
Backend API	PHP 8.3 + CodeIgniter 3.x (REST JSON API)
Database	MySQL 8.x – 56 tables, InnoDB, utf8mb4
Payment	Razorpay SDK (test + live mode)
Web Server	Apache 2.4 with mod_rewrite (.htaccess)
Dev Server	Node.js static file server (port 5500)
PHP Server	PHP built-in server (port 8000 for API)

3. System Architecture

```

graph TB
    subgraph CLIENT["🌐 Client (Browser – Port 5500)"]
        UI[index.html Entry Point]
        STORE[store.js – Reactive Store + canManage]
        ROUTER[app.js – Hash SPA Router]
        V1[views-students.js]
        V2[views-academics.js]
        V3[views-attendance.js]
        V4[views-exams.js]
        V5[views-finance.js]
        V6[views-staff.js]
        V7[views-operations.js]
        V8[views-reports.js]
        V9[views-settings.js]
    end

    subgraph API["🌟 REST API (CodeIgniter 3 – Port 8000)"]
        direction TB
        CI_ROUTER[index.php → CI Router]
        AUTH[Auth.php – Login/2FA/JWT]
        STUDENTS[Students.php]
        ACADEMICS[Academics.php]
        ATTENDANCE[Attendance.php]
        EXAMS[Exams.php]
        FEES[Fees.php]
        FINANCE[Finance.php]
        STAFF[Staff.php]
        OPERATIONS[Operations.php]
        RAZORPAY[Razorpay.php]
        SETTINGS[Settings.php]
        REPORTS[Reports.php]
    end

    subgraph DB["🗄️ MySQL 8 Database"]
        direction TB
        DB_USERS[(users)]
        DB_STUDENTS[(students)]
        DB_STAFF[(staff)]
        DB_FEES[(student_fees)]
        DB_ATTEND[(attendances)]
        DB_LIBRARY[(library_books / book_issues)]
        DB_EXAMS[(exams / exam_results)]
        DB_MISC[(45+ other tables)]
    end

    UI → STORE
    UI → ROUTER
    ROUTER → V1 & V2 & V3 & V4 & V5 & V6 & V7 & V8 & V9
    V1 & V2 & V3 & V4 & V5 & V6 & V7 & V8 & V9 →|"fetch() JSON API calls"| CI_ROUTER
    CI_ROUTER → AUTH & STUDENTS & ACADEMICS & ATTENDANCE & EXAMS & FEES & FINANCE & STAFF & OPERATIONS & RAZORPAY &
    AUTH & STUDENTS & ACADEMICS & ATTENDANCE & EXAMS & FEES & FINANCE & STAFF & OPERATIONS → DB_USERS & DB_STUDENTS

```

File Structure

```

Smart-School-Management-System/
├── frontend/                                ← SPA Frontend
│   ├── index.html                          ← Entry point (loads all scripts)
│   ├── css/
│   │   └── style.css                      ← Complete design system
│   ├── js/
│   │   ├── store.js                      ← Reactive store, canManage(), modal engine
│   │   ├── app.js                        ← SPA router, auth, API client, nav
│   │   ├── views-students.js             ← Students, Admission, TC, Credentials
│   │   ├── views-academics.js            ← Calendar, Downloads, Timetable, Live Classes
│   │   ├── views-attendance.js           ← Mark, QR, Report
│   │   ├── views-exams.js                ← Exams, Marks Entry, Admit Cards
│   │   ├── views-finance.js              ← Fees, Payroll, Accounts, Razorpay
│   │   ├── views-staff.js                ← Staff directory, Leave, Payslips
│   │   ├── views-operations.js           ← Library, Transport, Hostel, Notices
│   │   ├── views-reports.js              ← Analytics and Reports
│   │   └── views-settings.js             ← School settings, RBAC, Custom Fields
│   └── assets/
│       └── favicon.jpg
├── backend_codeigniter/                    ← REST API Backend
│   ├── application/
│   │   ├── controllers/                  ← 15 controllers
│   │   ├── models/                      ← 10 models
│   │   └── config/
│   │       ├── database.php              ← MySQL connection
│   │       └── routes.php                ← API route mappings
├── database/
│   └── smart_school_database_cpanel_2026.sql ← Full DB dump (56 tables)
├── index.php                              ← Apache root → CI bootstrap
├── .htaccess                              ← mod_rewrite rules
└── .env.example                            ← Environment config template
  
```

4. Application Working Flow

4.1 Authentication Flow

```

flowchart TD
    A([User Opens Browser]) --> B[Load index.html]
    B --> C[store.js loads → defines window.canManage]
    C --> D[app.js loads → defines auth, api, router]
    D --> E{Hash Route?}
    E --> F["#/Login or no hash"] --> F[Render Login Form]
    E --> G["#/dashboard etc"] --> G[localStorage user token?]
    G --> H[No token] --> F
    G --> I[Has token] --> H[Validate token with API]
    H --> J[Invalid] --> F
    H --> K[Valid] --> L[Load Dashboard]

    F --> M[User enters email + password]
    M --> N["POST /api/auth/login"]
    N --> O{2FA Enabled?}
    O --> P[Yes] --> Q[Show OTP Challenge]
    O --> R[No] --> M
    Q --> S["POST /api/auth/2fa-verify"]
    S --> T{OTP Valid?}
    T --> U[Yes] --> V[Store token + user in localStorage]
    T --> W[No] --> P
    V --> X[No] --> P
    P --> I

    I --> Y[Build sidebar nav based on role]
    Y --> Z[User navigates via sidebar/hash]
    Z --> AA[Router matches hash → renders view]
    AA --> AB["View calls api.get/post()"]
    AB --> AC["API attaches Bearer token header"]
    AC --> AD["CI Backend validates token"]
    AD --> AE[Return JSON data]
    AE --> AF[View renders DOM]
  
```

4.2 SPA Navigation Flow

```

flowchart LR
    A[User clicks nav item] --> B["window.location.hash = '#/route'"]
    B --> C["hashchange event fires"]
    C --> D["router() function in app.js"]
    D --> E["Matches route in routeMap"]
    E --> F["Calls viewFn() - async function"]
    F --> G["Fetches data from API"]
    G --> H["Returns HTML template string"]
    H --> I["document.getElementById('app').innerHTML = html"]
    I --> J["attachEventListeners()"]
    J --> K[UI is live and interactive]
  
```

5. Module Flowcharts

5.1 Fee Collection Flow (with Razorpay)

```

flowchart TD
    A([Admin/Accountant opens Fee Module]) --> B["Load student fee ledger via GET /api/fees"]
    B --> C["Payment mode?"]
    C --> D["Cash/Cheque/DD | D[Fill amount + receipt modal]"]
    D --> E["POST /api/fees/collect - offline"]
    E --> F["Generate PDF receipt"]
    F --> G["Mark Paid ✓"]

    C --> H["Online via Razorpay | H[\"POST /api/razorpay/create-order\"]"]
    H --> I["Razorpay order_id returned"]
    I --> J["Open Razorpay payment widget in browser"]
    J --> K["Payment successful?"]
    K --> L["Yes | L[\"Razorpay callback → POST /api/razorpay/verify\"]"]
    L --> M["Signature verification HMAC-SHA256"]
    M --> N["Valid | N[Update student_fees status=Paid]"]
    N --> O["Send WhatsApp receipt notification"]
    O --> G
    K --> P["Failed | P[Show error toast]"]
    P --> M
    M --> P
  
```

5.2 Library Book Issue & Return Flow

```

flowchart TD
    A([Librarian/Teacher opens Library]) --> B["GET /api/library/books"]
    B --> C["Display catalogue with stock counts"]
    C --> D["Action?"]

    D --> E["Add Book | E[Fill title, author, ISBN, qty modal]"]
    E --> F["POST /api/library/books"]
    F --> G["Insert into library_books"]
    G --> H["Update stat cards: Total, Available, Issued"]
    H --> C

    D --> I["Issue Book | I[Select book + student modal]"]
    I --> J["POST /api/library/issues"]
    J --> K["Insert into book_issues"]
    K --> L["UPDATE library_books SET available_qty = available_qty - 1"]
    L --> H

    D --> M["Return Book | M[\"POST /api/library/return {issue_id}\"]"]
    M --> N["UPDATE book_issues SET status=Returned, return_date=today"]
    N --> O["UPDATE library_books SET available_qty = available_qty + 1"]
    O --> H
  
```

5.3 Attendance Flow (Manual + QR)

flowchart TD

```

A([Teacher opens Attendance]) → B{Mode?}

B → |Manual| C["GET /api/students for class roster"]
C → D[Display student list with toggle buttons]
D → E[Teacher marks P/A/L/H for each student]
E → F["POST /api/attendance/save"]
F → G["Bulk INSERT into attendances table"]
G → H[Show attendance report cards]

B → |QR Code| I[Generate unique QR per student: ID + name]
I → J[Student scans QR with device]
J → K["POST /api/attendance/qr-scan"]
K → L["INSERT into qr_attendance_logs"]
L → M[Show live scan log on screen]

```

5.4 Exam & Marks Flow

flowchart TD

```

A([Admin creates Exam]) → B["POST /api/exams → INSERT into exams"]
B → C["POST /api/exams/schedule → INSERT exam_schedules"]
C → D[Exam scheduled with date, class, subject, room]
D → E["Print Admit Cards via GET /api/exams/admit-cards"]

E → F[Exam Date Arrives]
F → G[Teacher opens Marks Entry]
G → H["GET /api/students for class"]
H → I[Fill marks for each student per subject]
I → J["POST /api/exams/marks → INSERT exam_results"]
J → K[Auto-calculate grade + pass/fail]
K → L["GET /api/reports/performance"]
L → M[Generate performance analytics charts]

```

5.5 Student Admission Flow

flowchart TD

```

A([Receptionist/Admin]) → B[Open New Admission form]
B → C[Fill personal details: name, DOB, parent, class]
C → D[Fill custom fields: Blood Group, Allergies etc]
D → E["POST /api/students/admission"]
E → F["INSERT into students table"]
F → G["Generate Admission No: SS + Year + ID"]
G → H[Create login credentials: email + password]
H → I["INSERT into users table with role=student"]
I → J["Link students.user_id = users.id"]
J → K[Generate QR code for student ID card]
K → L[Assign to class, section, fee plan]
L → M[Admission Complete ✓]

```

6. Database ER Diagram

6.1 Core Entity Relationships

```
erDiagram
    users {
        bigint id PK
        string name
        string email
        string password
        string role
        string two_factor_secret
        timestamp created_at
    }

    students {
        bigint id PK
        bigint user_id FK
        string admission_no
        string name
        string class_name
        string section
        string father_name
        string mother_name
        string phone
        string email
        date dob
        string gender
        string blood_group
        string address
        date admission_date
        string status
    }

    staff {
        bigint id PK
        string emp_id
        string name
        string role
        string designation
        string email
        string phone
        string department
        decimal basic_salary
        string status
    }

    school_classes {
        bigint id PK
        string name
        string sections
        string class_teacher
        int room_no
    }

    subjects {
        bigint id PK
        string name
        string code
        string type
    }

    users ||--o| students : "has profile"
    users ||--o| staff : "has profile"
    students }o--|| school_classes : "enrolled in"
    school_classes }o--o{ subjects : "class_subjects"
```

6.2 Academic & Examination Relationships

```

erDiagram
    exams {
        bigint id PK
        string name
        string term
        string type
        string status
        date start_date
        date end_date
    }

    exam_schedules {
        bigint id PK
        bigint exam_id FK
        bigint class_id FK
        bigint subject_id FK
        date exam_date
        time start_time
        time end_time
        int total_marks
        int pass_marks
        string room
    }

    exam_results {
        bigint id PK
        bigint student_id FK
        string exam
        string subject
        int marks
        int total
        string grade
        string status
    }

    timetables {
        bigint id PK
        bigint class_id FK
        bigint subject_id FK
        string day
        time start_time
        time end_time
        string teacher_name
    }

    homeworks {
        bigint id PK
        bigint class_id FK
        bigint subject_id FK
        text description
        date due_date
        bigint assigned_by FK
    }

    exams ||--o{ exam_schedules : "has schedules"
    students ||--o{ exam_results : "has results"
    school_classes ||--o{ exam_schedules : "scheduled for"
    school_classes ||--o{ timetables : "has timetable"
    school_classes ||--o{ homeworks : "assigned to"

```

6.3 Finance & Fee Relationships

```

erDiagram
    fee_types {
        bigint id PK
        string name
        decimal amount
        string frequency
    }

    student_fees {
        bigint id PK
        string receipt_no
        bigint student_id FK
        string type
        decimal amount
        decimal paid
        string payment_mode
        string transaction_id
        decimal fine
        decimal discount
        string status
        string month
        string razorpay_order_id
        string razorpay_payment_id
        datetime payment_date
    }

    fee_discounts {
        bigint id PK
        bigint student_id FK
        bigint fee_type_id FK
        string discount_name
        enum discount_type
        decimal value
        string reason
    }

    transactions {
        bigint id PK
        string type
        string category
        decimal amount
        string description
        date transaction_date
        string reference_no
        string payment_mode
    }

    accounts_heads {
        bigint id PK
        string name
        enum type
        text description
    }

    students ||--o{ student_fees : "owes"
    students ||--o{ fee_discounts : "has discounts"
    fee_types ||--o{ fee_discounts : "applied to"
    fee_types ||--o{ student_fees : "categorises"

```

6.4 Library Relationships

```
erDiagram
    library_books {
        bigint id PK
        string title
        string author
        string isbn
        string category
        string publisher
        year publish_year
        string rack_no
        string language
        int qty
        int available_qty
        string status
    }

    book_issues {
        bigint id PK
        bigint book_id FK
        bigint student_id FK
        bigint staff_id FK
        string student_name
        date issue_date
        date due_date
        date return_date
        decimal fine_amount
        string status
    }

    library_books ||--o{ book_issues : "issued via"
    students ||--o{ book_issues : "borrows"
    staff ||--o{ book_issues : "borrows"
```

6.5 Operations Relationships

```

erDiagram
    transport_routes {
        bigint id PK
        string route_name
        string route_no
        string vehicle_no
        string driver_name
        int student_count
    }

    transport_stops {
        bigint id PK
        bigint route_id FK
        string stop_name
        string pickup_time
        decimal fare
    }

    student_transports {
        bigint id PK
        bigint student_id FK
        bigint route_id FK
        string stop_name
    }

    hostels {
        bigint id PK
        string name
        string type
        text address
        int intake
    }

    hostel_rooms {
        bigint id PK
        bigint hostel_id FK
        string room_no
        string type
        int capacity
        decimal fee
    }

    student_hostels {
        bigint id PK
        bigint student_id FK
        bigint hostel_id FK
        bigint room_id FK
        date allotment_date
    }

    transport_routes ||--o{ transport_stops : "has stops"
    transport_routes ||--o{ student_transports : "serves"
    students ||--o{ student_transports : "uses"
    hostels ||--o{ hostel_rooms : "has rooms"
    students ||--o{ student_hostels : "allotted to"
    hostel_rooms ||--o{ student_hostels : "accommodates"

```

6.6 Attendance & Communication Relationships

```

erDiagram
    attendances {
        bigint id PK
        bigint student_id FK
        bigint class_id FK
        date date
        enum status
        string remark
        bigint marked_by FK
    }

    staff_attendances {
        bigint id PK
        bigint staff_id FK
        date date
        enum status
        time time_in
        time time_out
    }

    qr_attendance_logs {
        bigint id PK
        string name
        string identifier
        enum person_type
        string status
        timestamp scanned_at
    }

    notices {
        bigint id PK
        string title
        text content
        date date
        string audience
        bool is_published
        bigint created_by FK
    }

    messages {
        bigint id PK
        bigint from_user_id FK
        bigint to_user_id FK
        string subject
        text body
        bool is_read
    }

    notification_logs {
        bigint id PK
        string recipient_name
        string recipient_contact
        enum channel
        string subject
        text body
        enum status
    }

    students ||--o{ attendances : "has records"
    staff ||--o{ staff_attendances : "has records"
    users ||--o{ notices : "creates"
    users ||--o{ messages : "sends"
    users ||--o{ messages : "receives"

```

6.7 RBAC Relationships

```
erDiagram
    permissions {
        bigint id PK
        string name
        string module
        string action
    }

    role_permissions {
        bigint id PK
        string role
        bigint permission_id FK
    }

    users {
        bigint id PK
        string role
        string email
        string name
    }

    permissions ||--o{ role_permissions : "granted via"
```

7. Role-Based Access Control (RBAC)

Role Permission Matrix

MODULE / ACTION	SUPER ADMIN	ADMIN	TEACHER	ACCOUNTANT	LIBRARIAN	RECEPTIONIST	PARENT	STUDENT
Students — View	✓	✓	✓			✓		
Students — Create/Edit/Delete	✓	✓				✓		
New Admission	✓	✓				✓		
Transfer Certificate	✓	✓						
Staff — View	✓	✓						
Staff — Create/Edit/Delete	✓	✓						
Mark Attendance	✓	✓	✓					
QR Attendance	✓	✓	✓			✓		
Attendance Report	✓	✓	✓					
Examinations	✓	✓	✓				✓	✓
Marks Entry	✓	✓	✓					
Admit Card	✓	✓	✓				✓	✓
Fee Collect / View	✓	✓		✓				
Razorpay Payments	✓	✓		✓				
Annual Calendar — View	✓	✓	✓				✓	✓
Annual Calendar — Add/Delete	✓	✓	✓				✗	✗
Download Center — View/Download	✓	✓	✓				✓	✓
Download Center — Upload/Delete	✓	✓	✓				✗	✗
Live Classes — View/Join	✓	✓	✓				✓	✓
Live Classes — Schedule/Delete	✓	✓	✓				✗	✗
Library — View	✓	✓	✓		✓			✓

MODULE / ACTION	SUPER ADMIN	ADMIN	TEACHER	ACCOUNTANT	LIBRARIAN	RECEPTIONIST	PARENT	STUDENT
Library — Issue/Return/Add	✓	✓	✓		✓			✗
Transport — View	✓	✓					✓	✓
Transport — Manage	✓	✓					✗	✗
Hostel — View/Manage	✓	✓						
Notices — View	✓	✓	✓				✓	✓
Notices — Create/Edit	✓	✓	✓					
Payroll — View	✓	✓		✓				
Payroll — Generate/Pay	✓	✓		✓				
Reports	✓	✓		✓				
Settings	✓							

RBAC Implementation

flowchart LR

```

A[User logs in] → B[JWT token stored in localStorage]
B → C["window.auth = auth (global)"]
C → D["window.canManage(['role1','role2'])"]
D → E{Read localStorage user.role}
E →|"super_admin / admin"| F["return true - full access"]
E →|"role in allowedRoles"| G["return true - role-specific access"]
E →|"role NOT in allowedRoles"| H["return false - UI element hidden"]
H → I["No button shown to user"]
G → J["Button/Feature rendered"]
F → J
  
```

8. API Architecture

Base URL & Structure

Backend: <http://127.0.0.1:8000/api/>
Frontend: <http://127.0.0.1:5500/frontend/>

Key API Endpoints

METHOD	ENDPOINT	CONTROLLER	DESCRIPTION
POST	/api/auth/login	Auth	Login with email + password
POST	/api/auth/2fa-verify	Auth	Two-factor OTP verification
POST	/api/auth/logout	Auth	Invalidate token
GET	/api/students	Students	List all students
POST	/api/students	Students	Create new student
PUT	/api/students/{id}	Students	Update student record
DELETE	/api/students/{id}	Students	Delete student
GET	/api/staff	Staff	List all staff members
POST	/api/attendance/save	Attendance	Bulk save attendance
POST	/api/attendance/qr-scan	Attendance	Log QR scan
GET	/api/attendance/report	Attendance	Attendance analytics
GET	/api/exams	Exams	List examinations
POST	/api/exams/marks	Exams	Save marks batch
GET	/api/fees	Fees	Student fee ledger
POST	/api/fees/collect	Fees	Record offline payment
POST	/api/razorpay/create-order	Razorpay	Create online payment
POST	/api/razorpay/verify	Razorpay	Verify payment signature
GET	/api/library/books	Operations	Book catalogue
POST	/api/library/books	Operations	Add new book
POST	/api/library/issues	Operations	Issue book to student
POST	/api/library/return	Operations	Return book
GET	/api/library/issues	Operations	Circulation ledger
GET	/api/transport/routes	Operations	Bus routes
GET	/api/hostel/rooms	Operations	Hostel rooms

METHOD	ENDPOINT	CONTROLLER	DESCRIPTION
GET	/api/calendar-events	Academics	Annual calendar
POST	/api/calendar-events	Academics	Add event
GET	/api/download-materials	Academics	Study materials
POST	/api/download-materials	Academics	Upload material
GET	/api/live-classes	Academics	Live class schedule
GET	/api/notices	Operations	Notice board
GET	/api/reports/dashboard	Dashboard	KPI dashboard stats
GET	/api/settings	Settings	School configuration
POST	/api/settings	Settings	Update settings
GET	/api/payroll	Finance	Staff payroll
POST	/api/payroll/generate	Finance	Generate salary

API Response Format

```
{
  "status": "success",
  "data": [...],
  "message": "Operation completed successfully",
  "total": 42
}
```

9. Module Catalogue (45 Modules)

Student Management

1. **Student Directory** — Full profile, search, filters
2. **New Admission** — Complete admission workflow with custom fields
3. **Login Credentials** — Generate username/password for students/parents
4. **Student Promotion** — Bulk class promotion at year end
5. **Transfer Certificate** — Automated TC generation
6. **Behavior Records** — Incident logging per student
7. **Student Documents** — Upload & manage certificates, birth cert, etc.

Staff Management

8. **Staff Directory** — Complete HR records
9. **Staff Attendance** — Daily time-in/time-out with status
10. **Leave Applications** — Leave approval workflow
11. **Payroll** — Salary generation, payslips, payment tracking

Academic Management

12. **Classes & Sections** — Class structure management
13. **Subjects** — Subject catalogue with class mapping
14. **Academic Sessions** — Year management (active session)
15. **Timetable** — Weekly class timetable per class
16. **Annual Calendar** — School events, holidays, exams
17. **Homework** — Assignment distribution per class/subject

Attendance

18. **Mark Attendance** — Manual per-student P/A/L/H
19. **QR Code Attendance** — Scan-based attendance with live log
20. **Attendance Report** — Class-wise and student-wise analytics

Examinations

21. **Examinations** — Create and schedule exams
22. **Marks Entry** — Subject-wise marks, auto-grade
23. **Admit Cards** — Student admit card generation and print
24. **Results** — Result cards with grade analysis

Finance

25. **Fee Types** — Tuition, Transport, Hostel, Library fees
26. **Fee Collection** — Per-student payment with receipt
27. **Fee Discounts** — Sibling, merit, and scholarship discounts
28. **Razorpay Online Payment** — Integrated online payment gateway
29. **Accounts / Transactions** — Income/expense ledger

30. **Financial Reports** — Revenue analytics and export

Library

- 31. **Book Catalogue** — Full inventory with stock tracking
- 32. **Book Issue** — Issue with due date and fine calculation
- 33. **Book Return** — Return and auto-increment available stock
- 34. **Circulation Ledger** — All issued/returned history

Operations

- 35. **Transport Routes** — Bus routes with stops and fares
- 36. **Hostel** — Building, rooms, student allotments
- 37. **Circular / Notices** — Published announcements per audience
- 38. **Noticeboard** — School-wide notice management

Digital Learning

- 39. **Download Center** — Study material upload and download
- 40. **Live Classes** — Google Meet / Zoom class scheduling and join

Reports & Analytics

- 41. **Dashboard KPIs** — Real-time summary cards
- 42. **Attendance Analytics** — Charts and export
- 43. **Financial Analytics** — Revenue, collection rates
- 44. **Academic Performance** — Student rank, grade distribution

System Administration

- 45. **School Settings** — Profile, campus, Razorpay config, WhatsApp, RBAC, Custom Fields
-

10. Key Design Decisions

10.1 SPA Architecture

The frontend is a **Single Page Application** using **hash-based routing** (`#/students` , `#/exams` , etc.) with no framework. This decision was made for:

- Zero build step needed — works as static files
- Easy cPanel deployment (no Node.js server needed for frontend)
- Fast navigation without page reloads

10.2 Reactive Store

`window.SS_STORE` provides localStorage-backed reactive state that:

- Persists data across page refreshes as a fallback to the API
- Allows optimistic UI updates before API confirms
- Reduces redundant API calls with in-memory caching

10.3 Token-Based Auth with 2FA

- JWT-style Bearer token stored in `localStorage`
- All API requests attach `Authorization: Bearer <token>` header
- Optional TOTP-based two-factor authentication for admin accounts

10.4 Offline Fallback Strategy

Each view function follows this pattern:

1. Try to fetch from MySQL via REST API
2. On API failure → fall back to `SS_STORE` localStorage data
3. UI always renders — never blank due to API failure

10.5 Razorpay Payment Integration

- Server creates `order_id` via Razorpay API (backend secret key protected)
 - Frontend opens Razorpay checkout widget
 - On success, `payment_id` + `signature` sent to backend for HMAC-SHA256 verification
 - Only after verification is `student_fees.status` updated to `Paid`
-

11. Deployment Architecture

```
graph TD
    subgraph INTERNET["🌐 Internet"]
        BROWSER["Student / Parent / Teacher Browser"]
    end

    subgraph CPANEL["🖥️ cPanel Shared Hosting / VPS"]
        APACHE["Apache 2.4\n(.htaccess mod_rewrite)"]
        PHP["PHP 8.3 + CodeIgniter 3"]
        MYSQL["MySQL 8.x\nDatabase: smart_school"]
        FILES["Static Frontend Files\n/public_html/frontend/"]
    end

    subgraph EXTERNAL["🔌 External Services"]
        RAZORPAY_EXT["Razorpay Payment Gateway"]
        WHATSAPP_EXT["WhatsApp Business API"]
        ZOOM["Zoom / Google Meet"]
    end

    BROWSER -->|HTTPS| APACHE
    APACHE -->|"Static: /frontend/"| FILES
    APACHE -->|"API: /api/"| PHP
    PHP -->|SQL Queries| MYSQL
    PHP -->|Payment API| RAZORPAY_EXT
    PHP -->|WhatsApp notification| WHATSAPP_EXT
    BROWSER -->|Join class link| ZOOM
```

Environment Configuration (`.env`)

```
DB_HOST=localhost
DB_DATABASE=smart_school
DB_USERNAME=smart_school_user
DB_PASSWORD=<secure_password>

RAZORPAY_KEY_ID=rzp_live_XXXXXXX
RAZORPAY_KEY_SECRET=XXXXXXXXXXXXXXXXX
RAZORPAY_MODE=live

WHATSAPP_NUMBER=+919876543210
```

Appendix: Complete Database Table Index

#	TABLE NAME	PURPOSE
1	users	System user accounts & authentication
2	students	Student profiles & admission records
3	staff	Employee HR records
4	school_classes	Class & section configuration
5	subjects	Subject catalogue
6	class_subjects	Class ↔ Subject mapping
7	academic_sessions	Academic year management
8	timetables	Weekly class schedules
9	homeworks	Teacher-assigned homework
10	attendances	Student daily attendance
11	staff_attendances	Staff daily attendance
12	qr_attendance_logs	QR scan attendance log
13	exams	Examination master records
14	exam_schedules	Per-class exam schedule
15	exam_results	Student marks & grades
16	fee_types	Fee category definitions
17	student_fees	Individual fee records & payments
18	fee_discounts	Scholarship & concession records
19	transactions	General income/expense ledger
20	accounts_heads	Chart of accounts
21	salary_records	Monthly payroll records
22	payslip_items	Individual payslip line items
23	library_books	Book catalogue & inventory
24	book_issues	Book issue & return ledger
25	transport_routes	Bus route master
26	transport_stops	Bus stop schedule
27	student_transports	Student ↔ Route assignment
28	hostels	Hostel building records
29	hostel_rooms	Room configuration

#	TABLE NAME	PURPOSE
30	student_hostels	Student room allotment
31	calendar_events	Annual academic calendar
32	download_materials	Study material uploads
33	live_classes	Live online class schedule
34	notices	School circular notices
35	messages	Internal staff messaging
36	notification_logs	SMS/Email/Push log
37	leave_applications	Staff & student leave requests
38	permissions	Permission definitions (44 permissions)
39	role_permissions	Role ↔ Permission mapping
40	custom_fields	Dynamic admission form fields
41	student_custom_field_values	Custom field data per student
42	student_documents	Scanned documents per student
43	student_promotions	Year-end promotion records
44	student_siblings	Sibling relationship for discounts
45	student_timelines	Student academic timeline events
46	student_notes	Teacher notes on students
47	school_settings	System-wide configuration
48	personal_access_tokens	Bearer token store
49	password_reset_tokens	Password reset links
50	sessions	PHP session storage
51	cache	Application cache store
52	cache_locks	Cache lock management
53	jobs	Background job queue
54	job_batches	Batch job tracking
55	failed_jobs	Failed job archive
56	migrations	Database version tracking

Report generated: September 29, 2026 | Infosof Technologies | Smart School Management System v1.5